

JAVA PROGRAMMING

(R2025) – Exam-Oriented Important Notes

Concise exam-oriented notes covering the major Java programming concepts, with definitions, key features, comparisons and important exam points.

UNIT I – JAVA FUNDAMENTALS

Java: Java is a high-level, class-based, object-oriented programming language designed to be portable and secure.

Features: Simple, object-oriented, platform independent, secure, robust, portable, multithreaded and distributed.

JDK, JRE and JVM: JDK provides tools for Java development; JRE provides the environment to run Java programs; JVM executes Java bytecode.

Bytecode: Java source code is compiled into bytecode, which can run on any suitable JVM.

Data Types: Primitive types include byte, short, int, long, float, double, char and boolean. Reference types include classes, arrays and interfaces.

Variables: Variables may be local, instance or static/class variables.

Operators: Arithmetic, relational, logical, assignment, unary, bitwise, conditional and shift operators are commonly used.

Control Statements: if, if-else, nested if, switch, for, while, do-while, break and continue.

Arrays: An array stores multiple values of the same type. Java supports one-dimensional and multidimensional arrays.

Exam focus: Explain Java features; compare JDK/JRE/JVM; explain bytecode; write programs using arrays and control statements.

UNIT II – CLASSES, OBJECTS AND INHERITANCE

Class: A class is a blueprint that defines data members and methods.

Object: An object is an instance of a class and represents a real-world entity or program component.

Constructor: A special member used to initialize objects. It has the same name as the class and has no return type.

Method: A method contains a set of statements that performs a particular operation.

Encapsulation: Binding data and methods together in a class and controlling access using access modifiers.

Inheritance: Mechanism by which a class acquires properties and behaviour from another class. Common forms include single, multilevel and hierarchical inheritance.

Method Overloading: Same method name with different parameter lists; it supports compile-time polymorphism.

Method Overriding: A subclass provides a specific implementation of an inherited method; it supports runtime polymorphism.

super and this: *this* refers to the current object; *super* refers to members of the immediate parent class.

Exam focus: Explain class/object, constructors, encapsulation, inheritance, overloading and overriding with suitable programs.

UNIT III – INTERFACES, PACKAGES AND EXCEPTION HANDLING

Interface: An interface defines a contract of methods that implementing classes must provide. It supports abstraction and multiple-type inheritance.

Abstract Class: A class declared abstract that may contain abstract and concrete methods. It cannot normally be instantiated directly.

Package: A package groups related classes and interfaces and helps organize code and control access.

Access Modifiers: public, protected, default and private control visibility of classes and members.

Exception: An abnormal condition that interrupts the normal flow of program execution.

Exception Handling: Java uses try, catch, finally, throw and throws to handle exceptional conditions.

Checked and Unchecked Exceptions: Checked exceptions are generally checked at compile time; unchecked exceptions occur at runtime.

Custom Exception: A programmer-defined exception can be created by extending Exception or an appropriate subclass.

Exam focus: Compare abstract class and interface; explain packages and access control; explain try-catch-finally, throw and throws.

UNIT IV – STRING, COLLECTIONS AND GENERICS

String: String represents a sequence of characters. String objects are immutable.

StringBuffer and StringBuilder: Mutable character-sequence classes used when repeated string modifications are required.

Collections Framework: Provides reusable interfaces and classes for storing and manipulating groups of objects.

List: Ordered collection that can contain duplicates. Common implementations include ArrayList and LinkedList.

Set: Collection that does not allow duplicate elements. Common implementations include HashSet and TreeSet.

Map: Stores key-value pairs. Common implementations include HashMap and TreeMap.

Iterator: Used to traverse collection elements sequentially.

Generics: Provide type safety by allowing classes, interfaces and methods to work with specified data types.

Exam focus: Compare String/StringBuilder/StringBuffer; explain List, Set and Map; write collection programs; explain generics.

UNIT V – MULTITHREADING, I/O AND FILE HANDLING

Thread: A lightweight unit of execution within a program. Multithreading allows multiple tasks to execute concurrently.

Creating Threads: A thread can be created by extending Thread or implementing Runnable.

Thread Life Cycle: Common states include new, runnable, running, blocked/waiting and terminated.

Synchronization: Controls access to shared resources so that concurrent threads do not produce inconsistent results.

Inter-thread Communication: Java provides mechanisms such as wait(), notify() and notifyAll().

Streams: Java I/O uses streams to read and write data. Byte streams handle binary data; character streams handle text data.

File Handling: Java provides classes such as File, FileReader, FileWriter, BufferedReader and BufferedWriter for file operations.

Serialization: Converts an object's state into a byte stream so it can be stored or transmitted.

Exam focus: Explain thread creation and life cycle; synchronization; byte vs character streams; file handling and serialization.

UNIT VI – JDBC, GUI AND MODERN JAVA CONCEPTS

JDBC: Java Database Connectivity is an API used to connect Java applications with databases and execute SQL statements.

JDBC Steps: Load/register driver when required, establish connection, create statement/prepared statement, execute SQL, process results and close resources.

PreparedStatement: Represents a precompiled SQL statement and is useful for parameterized queries.

ResultSet: Represents data returned by a query and provides methods to access rows and columns.

Lambda Expression: A concise way to represent a function-like operation, commonly used with functional interfaces.

Functional Interface: An interface containing one abstract method; it can be used with lambda expressions.

Stream API: Supports declarative processing of collections using operations such as filter, map and collect.

GUI: Java GUI applications can be built using libraries such as AWT and Swing. Components include frames, buttons, labels and text fields.

Exam focus: Explain JDBC architecture/steps; Statement vs PreparedStatement; ResultSet; lambda expressions, streams and basic GUI components.

LAST-MINUTE REVISION CHECKLIST

1. Learn the definitions and features of Java, JVM, JDK, JRE and bytecode.
2. Practice programs using classes, constructors, inheritance, overloading and overriding.
3. Prepare comparisons: abstract class vs interface, overloading vs overriding, checked vs unchecked exceptions.
4. Practice String and Collections: ArrayList, LinkedList, HashSet, TreeSet and HashMap.
5. Practice thread creation, synchronization and basic file-handling programs.
6. Remember JDBC steps and important objects: Connection, Statement/PreparedStatement and ResultSet.
7. Revise lambda expressions, functional interfaces and Stream API.
8. For long answers, write: definition → syntax/steps → explanation → example/program → advantages/applications → conclusion.